Computer Games Development
Project Crescent Rise Report
Year IV

David Strikaitis
C00283152
13/04/2026

Contents

Acknowledgements 3
Project Abstract 4
Project Introduction and/or Research Question 5
Literature Review 6
Evaluation and Discussion 7
Project Milestones 8
Major Technical Achievements 10
Project Review 11
Conclusions 12
Future Work 12
References 12

Acknowledgements

I would like to thank the following people who assisted in completing this project including:

Dr. Noel O'Hara - who gave many insightful ideas and thought on the project as a whole
All the supervisors who gave me some good ideas on what to do with the gameplay, and what direction I should take with the planets

Project Abstract

This project is a procedural planet generation system, with an integrated level editor. The project is built entirely in C++ using OpenGL for rendering and using some small libraries; such as ImGui for interface development, and GLM for mathematics. The entire system was built from the ground up relying solely on low level libraries, and no game engine. This provides complete control over rendering, generation and performance, resulting in a computationally efficient system, and technical robustness due to the absence of external dependencies.

The problem is that planet generation is extremely complicated, and making a traversable mesh is significantly more complex and technical. The generation of high-fidelity explorable planets is long and demanding, and the creation of new, unique and interesting planets is even more technically challenging. A 3D world that is random and fully explorable is highly appealing from both a gameplay perspective and technical perspective.

To address these problems, I explored the creation of randomized planets based on a set of pre-defined parameters, making planets unique and interesting every time. These follow certain rules as to prevent unplayable planets.

The key technical features are vast, which functions as a fully unique and independent game engine for the creation of custom explorable planets which are traversable through physical interaction. The creation of a custom level editor integrates the game engine into the making of unique gameplay for every playthrough.

In the end I created a highly efficient game engine allowing for many weapons, enemies, unique game objects, and of course planets. The game allows for novel interaction with enemies on a fully unique planet. This reflects how many full-scale AAA titles produce interesting environments through the use of procedural generation, this is tied together through the use of real-time gameplay mechanics to create a polished experience.

This project is a demonstration of my technical competence in the creation and manipulation of low-level graphics programs. It shows the different use cases of procedural generation, and the creation of fully dynamic game worlds. This project has potential to be extended to be release as a fully-fledged commercial title, using the procedural development as the core of the game.

Project Introduction and/or Research Question

The production of game environments has developed over the years, moving from hand crafted environments to procedural generation, which has become very popular in gaming. Procedural generation is the creation of environments through code; this technique was popularized by titles like Minecraft. Procedural generation allows for a rule-based approach for creating many environments quickly.

Due to the popularity of games like Minecraft, Procedural generation has become a focal point of discussion in the gaming community. There are many different approaches to the creation of planets. I decided to make my own, as I believe that many games explore the creation of 3D environments but are

often built on top of a game engine. This made me decide to explore the intricacies of creating 3D worlds from scratch and making a game engine tailored towards the optimisation and creation of 3D planets.

I realised that the creation of 3D environments, and games is much more technically challenging and fascinating than any other idea I had produced. Scrapped ideas included simple 2D generation, or non-planet like creations. The creation of unique, interesting, and playable planets became a central idea for my project.

I decided to explore the idea of making an explorable 3D planet made from scratch in C++ and OpenGL within a year long time. I also explored the idea of implementing a level editor, allowing for any player to create variations to the planets, without using any code. This also begged the question; how can you make this entire editor work efficiently on any machine. This helped me decide a research question.

Can a simple 3D game engine be created to build planets procedurally, and have the planets editable by the player, as well as interactable and traversable in C++ and OpenGL?

This research question is a good basis as it also allows for future extensibility and the potential of creating a full title in future. This project aims to create this game engine, create these planets, and make it engaging, and customisable by the player.

Literature Review

The area of procedural generation is not a new one and has been around in the context of computing for many years. This has also been popularized by video games as they provide a practical application for the mathematical theories of procedural maths.

In gaming in specifically the area of space exploration some games have become some of the most prominent games in history thanks to the idea of procedural generation. Games such as Elite Dangerous which has 1:1 Scale of

the Milky Way, and No Man's Sky which has a near infinite number of planets thanks to procedural generation are some of the biggest inspirations for my project.

There are many approaches to procedural generation such as search based procedural generation which tries many different solutions, until a suitable solution is found (Ian L. McGathey 2014 [1]). Rule based generation which is what ended up using for my project. Which allows for the setting of predefined rules for the content of generation, after which the rest is random (Chin Kim On et al. 2017[2]). These are the main 2 ways in which content can be procedurally generated which are applicable to my project (Also using machine-based algorithms but this doesn't suit my project)

The popularization of realistic planets through Elite Dangerous and No Man's Sky, show the practicality of creating spherical planets, procedurally. For this reason, I tailored my project around doing this, in my own engine.

Evaluation and Discussion

In the end the goal of this project was fulfilled. The plan to make traversable, unique editable planets is now possible. The game engine / planet editor is fully functional and has achieved the intended goals that were set. The level editor stores the levels to a JSON and loads them on each startup, making every game unique. During gameplay this resulted in some very interesting terrain every run, but it also makes some very difficult planets to traverse on occasion, large island based planets may come with one island that may be too far to reach, while this is a difficult thing to play, as a system it is still very fun and unique.

Surface instancing became an issue throughout development. When too many instances were placed on the surface the game would lag heavily or sometimes even fail to load. The instancing system was modified to support editable levels of quality in the code; however this stayed an issue, I added distance culling to help combat this however, which improved the game's frame rate.

The level editor has many parameters it can modify such as enemies, surface instances, planet quality, planet generation method etc. The main drawback of the editor is that it may be difficult to use for anyone that hasn't used it before. It also could lead to the user breaking something as there are no restrictions on the settings, and the user could theoretically make something way too high quality and crash the game. Other than this, it allows for full editing capabilities of the planet itself without the need to modify the code.

The gameplay system while fun mechanically is very flawed. Outside of the basic running around and shooting targets, as there is no goal, score or story. This makes the system lack long term progression. Outside of the ‘cool’ factor of allowing you to traverse the planet, there is little justification for gameplay. However the gunplay is fun and intuitive, the enemies that are there (2 enemy types) allow for some interesting interactions, with their unique movement and abilities. The hit highlights and enemy glow on attacks are all incredibly fun and intuitive, communicating with the player just enough information that keeps the difficulty, but doesn't subtract from gameplay itself.

The main limitation on this system is gameplay, making it unattractive for the average user to play. Unfortunately, this project is lacking as a full gameplay experience, however, as a technical project, this still has a lot of value.

Project Milestones

Development of my project began as soon as I developed the initial concept, this was in September in 2025. I began with some project structure, including Game Objects and abstract script classes. I also came up with a rendering pipeline for OpenGL, As I hadn't worked with it before. The research, setup and development in the early stage took an exceedingly long time. This phase was from the end of September to early November. This was unfortunately the only way as I had to take the time to understand how 3D objects were rendered, how to colour them and how to manipulate them programmatically. This was unfortunately about a month longer than I would have liked, as this meant no Planets were generated at this point.

To make planets I decided to start with a flat terrain. I made some noise functions and made terrain that was manipulated randomly by the noise, believing this would transfer to a spherical planet. This was not transferable; due to that I spent the rest of November on this.

Late November to early December saw me create planets that were manipulated by noise functions, during this time I also made a moving character and an online server-based connection for multiplayer, this worked well at the time and allowed for a full multiplayer experience, however this was scrapped due to time complexity in development.

During December to early February, I made the planet surface more interesting, the player could move on it, there was interesting water, and there were the beginnings of surface instancing. At the same time I started work on the level editor, this time period had probably the most errors, as there were crashes, and such I spent a lot of time fixing errors, and even at this point some errors took multiple days to solve due to the decoupling between game objects (one ampersand for instance took 3 days to find was missing).

The final phase of development was February to April. This time was the gameplay and polish phase, main menus, weapons, enemies and much more were the focus during this time. I also had many interesting errors and setbacks during this phase, many of which were due to the nature of the project, for instance the enemy didn't track the surface position properly so would clip through the floor. This was by far the most interesting phase of development however, as I had a good grasp on OpenGL and how I could edit the Game engine to do as I needed at this point.

Throughout the project I feel I was behind of schedule mainly due to the complexity of the low level programming I was dealing with, OpenGL is very complex, and took longer than expected to get the hang of, outside of this the

project was on schedule, and was just about finished for the submission deadline.

Major Technical Achievements

Game Engine / framework. The game engine that was built from the ground up using just C++ and OpenGL. This gave me control of all the resources used, allowing for optimisations where needed. The game engine like architecture includes a game object system, abstract script classes, decoupled send Message system and tag-based object retrieval, all mimicking how other game engines function.

Procedural Planet generation through specific rules, allowing for custom and unique planets, for as many different variants as the developer desires. This is done using a noise deformation on an Ico sphere (originally a cube sphere which was scrapped).

Planet rendering through height-based colouring allowing for variations based on the planet's height. This makes the terrain realistic for the player who can traverse it on foot.

Planet editor allowing for real time in game editing, JSON save and load pipeline for planet retrieval and storage. Also, a random planet loading feature for replayability by the user. This all comes together to make a coherent level editor for the user who can make custom planets, unique and interesting just using the game, and can keep these planets for later games.

Planet surface instancing allows for procedural placementof objects on the planet's surface, for Instance grass can be placed to make the planet look more realistic, this is also saved, making the planet editor even more useable. Simple distance culling was implemented for the optimisation of the placed objects to increase the fps of the user.

Many gameplay system abstractions were implemented. Such as a weapon abstraction allowing for the quick creation of new unique weapons with just adding a few lines of code. Enemy state abstraction with a custom finite state machine. Interaction system abstraction for picking up items with just a single line of code allowing any object to be picked up.

Project Review

I feel a lot of things went quite right, I ended up exploring my game idea in the correct direction, although slow I believe that the pace of the project was okay. The biggest pacing issue that I faced was the beginning with learning OpenGL and getting accustomed to its rendering pipeline, and all its custom functions. This ended up taking over a month to figure out how to configure things to render efficiently.

This was the biggest issue by far, learning OpenGL and making all the parts of it work properly together and getting the specific function names to things I wanted done.

There are unfortunately some things I didn't get fully completed in time for the deadline. These included adding some more UI elements to help the player, such as a reticule ammo count etc. I also wanted to add a boss enemy which would have unique patterns and would allow the player to go to the next planet after defeating the boss. I also wanted to add some more elements such as more particles hit effects, and enemy health bars that would display above them in the world. I also wanted to expand on the interaction system allowing for the pickup of weapons and ammo, and other area specific items. I didn't have time for these and that is the only reason they could not be implemented.

I think that my technology choices for this project were solid, although OpenGL was complicated to learn at first it gave me a good understanding of how things function and behave across the board in 3D development, it also allowed me to explore all the intricacies of developing from the ground up in my own unique environment. The main implication of choosing OpenGL / C++ was that the bar of entry was extremely high from a technical standpoint,

which consumed a lot of my early time and made the project complex as a whole.

If someone were to attempt this project, or I started again, I'd tell them to ensure they understand OpenGL. Due to its complexities, they should either understand how it works, or at the very least know what they are in for with it, as it is a complicated library, that does no hand holding whatsoever like other libraries would.

Conclusions

In conclusion, I think my project went very well. All major features were implemented using OpenGL and C++, this all allowed for immense control over every aspect of the rendering pipeline, the interaction pipeline, and even inputs and updates. I learned a lot about how 3D works, how meshes work, how animations work, how shaders work, how 3D objects work, how to manipulate them and so much more.

I believe this project is unfortunately unfinished due to what I listed in the Project review, but even without those features the projects' technical achievements are exhaustive, and the implementation of them taught me so much on how to use OpenGL.

Future Work

A fully fledged environmental editor would be interesting to explore. Also making a proper game engine, with actual editor capabilities like unity would be a great thing to try. It would be great to explore and would have many technical interesting and advanced things to look out for.

References

[1] Ian L. McGathey 2014 Title: *Search Based Procedural Generation in Games.* Accessible here: Mcgathey.pdf (Accessed 10/04/2026)

[2] Chin Kim On, Ng Wai Foong, Jason Teo, Ag Asri Ag Ibrahim, and Tan Tse Guan 2017 Title: *Rule-based Procedural Generation of Item in Role-playing*

Game. Accessible at: Microsoft Word - 17. Chin Kim On - 1739 (Accessed 10/04/2026)